Not Ready Yet: An Industrial Assessment of Coding Agents for Feature Implementation and the Promise of Harness Engineering

Zihan Wu
Huawei Technologies Co., Ltd.
Shenzhen, China
wzh4464@gmail.com

Abstract

LLM-based coding agents now resolve over 70% of SWE-bench bug-fix tasks, prompting industry interest in deploying them for larger-scale feature implementation. Yet no rigorous evaluation exists for multi-file feature development in industrial codebases. We present an empirical assessment of four agent configurations—Claude Opus 4.6, GPT-5.4 (Codex), Cursor Composer-2, and GLM-5.1 (OpenCode)—across 12 feature-implementation tasks drawn from four production repositories (Huawei CANN, MindSpeed, torch_npu, and Kubernetes) in C++, Python, Go, and AscendC, yielding 96 experiments each independently scored by three LLM evaluators for cross-validation. The best configurations achieve 12.5% PASS under our rubric, and overall only 8.3% of experiments produce production-equivalent implementations. Task complexity exposes an absolute capability cliff: top agents pass 25.0% of low-complexity tasks but 0% of high-complexity ones. Root-cause analysis reveals that 84.1% of failures stem from engineering-addressable gaps—primarily incomplete file coverage—rather than fundamental model-capability limitations. Detailed prompts improve pass rates by up to 25.0 percentage points for capable models, confirming prompt granularity as the single strongest performance lever. Our findings indicate that coding agents are not ready for autonomous deployment on feature implementation, but that structured specifications and process-side interventions offer a practical direction to bridge the gap.

ACM Reference Format:
Zihan Wu. 2026. Not Ready Yet: An Industrial Assessment of Coding Agents for Feature Implementation and the Promise of Harness Engineering. In . ACM, New York, NY, USA, 8 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Software engineering teams are rapidly integrating AI into their development workflows. Industry surveys report that over 80% of professional developers now use AI-assisted coding tools [6], and frontier coding agents—autonomous systems that navigate codebases, write code, and run tests—have demonstrated remarkable progress on established benchmarks. On SWE-bench Verified, the most widely cited evaluation for coding agents, top systems now resolve over 70% of tasks [7]. These results have generated significant enthusiasm for deploying coding agents on increasingly ambitious engineering tasks, including full feature implementation in enterprise codebases.

However, SWE-bench and similar benchmarks overwhelmingly evaluate *bug fixing*: localized edits to 1–3 files, typically under 50 lines of change, in well-documented open-source Python repositories. Feature implementation in industrial settings is a fundamentally different class of task. It requires API design, feature gate lifecycle management, cross-module coordination, backward compatibility constraints, test suite construction, and adherence to project-specific conventions—engineering activities that demand architectural reasoning, not just code generation. A Kubernetes feature gate, for example, touches API types, validation, defaulting, conversion, controllers, and dozens of generated files across 50–100 files; a CANN operator implementation requires host code, device kernels, tiling engines, and framework integration layers spanning C++, AscendC, and Python. No systematic evaluation exists for how coding agents perform on this class of tasks in production codebases.

We present the first industrial-scale capability assessment of coding agents on feature implementation tasks. Our benchmark comprises 12 tasks drawn from four production repositories—Kubernetes (Go), CANN operator kernels (C++/AscendC), MindSpeed training framework (Python), and torch_npu (Python)—spanning three complexity tiers from single-file fixes to 98-file architectural changes. We evaluate four commercially available agent configurations across 96 experiments—each independently scored by three LLM evaluators for cross-validation—and find a clear answer: **Not Ready Yet** for autonomous deployment. The best configurations (Claude Opus 4.6 and Codex GPT-5.4) each achieve a 12.5% PASS rate under our rubric; the overall rate across all configurations is 8.3%. Even restricting to the most favorable condition—the best agents with detailed implementation specifications—the pass rate reaches only 25.0%, with critical gaps in test coverage, API design coherence, and cross-module completeness.

This paper makes five contributions:

(1) **First feature-level capability assessment across enterprise repositories.** We design and execute a controlled

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

evaluation of coding agents on 12 real feature implementation tasks spanning four languages and three complexity tiers, using merged production code as ground truth (§2).

(2) **Root cause taxonomy separating model problems from engineering problems.** Analysis of 88 non-PASS experiments yields a four-category failure taxonomy. The dominant failure mode is incomplete file coverage (49% of failures), not structural mismatch (1%), and 84.1% of failures are engineering-addressable—the agent understands the problem but fails to cover the full scope (§4).

(3) **Quantified prompt engineering effects.** We measure the effect of input-side specification detail through controlled prompt comparisons: detailed specifications improve PASS rates by up to 25.0 percentage points for capable agents and composite scores by +2.2 to +2.6/15, confirming prompt granularity as the single strongest performance lever (§5).

(4) **Complexity-driven completeness cliff.** On high-complexity tasks (10–98 ground-truth files), 88% of non-PASS experiments are critically incomplete ($B < 2$), compared to 37% on low-complexity tasks. File coverage data shows up to 30× differences between agents on the same task (§4).

(5) **Harness engineering roadmap.** We propose a phased adoption roadmap from language-aware validation through fix-cycle loops to architecture-aware planning, with estimated impact ranges grounded in our empirical gap analysis (§5.3).

The remainder of this paper is organized as follows. Section **??** describes the industrial context and motivation. Section 2 presents the study design, including task selection, agent configurations, and evaluation framework. Section 3 reports the main results. Section 4 analyzes root causes of failure. Section 5 examines harness engineering effectiveness and limitations. Section 6 surveys related work, Section 7 discusses threats to validity, and Section 8 concludes.

2 Study Design

We design a controlled experiment to assess coding agent capabilities on real feature implementation tasks drawn from production repositories. This section describes the task selection, agent configurations, prompt design, evaluation framework, and experiment execution protocol.

2.1 Task Selection

We select 12 tasks from four open-source repositories that are actively maintained by Huawei and the broader community. Each task corresponds to a merged pull request or commit that implements a complete feature or bug fix. Table 1 summarises the task registry.

Selection criteria. Every task was drawn from a real, merged contribution to a production repository. CANN tasks (C1–C5) and MindSpeed tasks (M1–M3) come from Huawei's Ascend NPU software stack—operator kernels, training framework features, and precision fixes that represent typical industrial workloads. Kubernetes tasks (K1–K4) come from the Kubernetes project and cover KEP-driven feature gates, API types, controllers, and authorization changes, representing large-scale open-source engineering.

Table 1: Task registry. GT = ground truth (merged code). Complexity is classified by file count, cross-component coordination required, and domain-specific knowledge.

ID	Repository	Complexity	Lang	GT Files	GT Lines
C1	cann-ops-adv	Low	C++	1	3
C2	cann-ops	Low	C++	4	9
M1	MindSpeed	Low	Python	3	19
K1	kubernetes	Low	Go	13	1,010
C3	torch_npu	Medium	Python	9	637
C4	cann-ops	Medium	C++/AscendC	24	1,273
M2	MindSpeed	Medium	Python	6	150
K2	kubernetes	Medium	Go	35	3,759
K3	kubernetes	Medium	Go	49	11,000
C5	cann-ops	High	C++/AscendC	27	3,372
M3	MindSpeed	High	Python	10	948
K4	kubernetes	High	Go	98	6,507

Table 2: Agent configurations evaluated on the full 12-task benchmark. Each runs all $12 \times 2 = 24$ task–prompt combinations.

Config	Model	Framework	N
Claude	Claude Opus 4.6	Claude Code	24
Codex	GPT-5.4	Codex CLI	24
Cursor	Composer-2	Cursor Agent	24
OpenCode	GLM-5.1	OpenCode	24

Complexity classification. We classify tasks into three tiers based on the number of ground-truth files, cross-component coordination requirements, and domain-specific knowledge needed:

- *Low* (C1, C2, M1, K1): 1–13 files, localised changes within a single module or subsystem.
- *Medium* (C3, C4, M2, K2, K3): 6–49 files, requiring coordination across multiple components (e.g., host and device code, API types and controllers).
- *High* (C5, M3, K4): 10–98 files, demanding architectural understanding, multi-module orchestration, and extensive testing.

The tasks span four languages (C++, AscendC, Python, Go) and three complexity tiers, providing a diverse substrate for evaluating agent capabilities across different engineering contexts.

2.2 Agent Configurations

We evaluate four primary agent configurations, summarised in Table 2. Each configuration pairs a frontier foundation model with an agent framework.

Product-level comparisons. Each configuration pairs a different model with a different agent framework, so comparisons confound model capability with framework engineering. These comparisons are best understood as *product-level* assessments—comparing the capabilities of commercially available agent tools—rather than controlled model ablations. We discuss this confound further in §7.

2.3 Prompt Design

Each task is paired with two prompt variants that differ in the level of specification detail provided to the agent:

- **Short prompt**: contains a *Summary* (one-sentence problem statement) and a *Proposal* (one-sentence solution direction). Typically 2–4 sentences.
- **Long prompt**: extends the short prompt with *Motivation* (why the change is needed) and *Design Details* (numbered list of implementation considerations). Typically 15–30 sentences.

Design principle. Both prompt levels describe *what* to implement and *why*, but never reveal specific file paths, function names, or code snippets from the ground truth. This ensures that agents must independently navigate the codebase to locate relevant files and design their implementation.

Example (C1, buffer alias fix). The short prompt reads: "*ScaledMaskedSoftmaxGradV2 has a tensor lifetime violation in ComputeSoftmaxGrad—a buffer is used after its queue resource is freed. Replace the reference with an equivalent surviving buffer.*" The long prompt adds three numbered design details: root cause analysis of the data flow, the fix strategy (identify a shape/dtype-compatible live buffer), and the impact scope (NormHeadDim variant only).

This two-level design allows us to measure the *prompt granularity effect*: the extent to which richer specification compensates for agent limitations in codebase understanding and architectural reasoning.

2.4 Evaluation Framework

Ground truth. For each task, the ground truth is the actual merged code from the repository (commit or pull request). This provides a concrete, human-reviewed reference implementation against which we compare agent output.

Scoring dimensions. Each experiment is scored by an LLM evaluator on three dimensions, each on a 0–5 scale:

- **Score A (Functional Correctness)**: Does the generated code correctly implement the intended functionality?
- **Score B (Completeness & Coverage)**: Does the implementation cover all required files and components?
- **Score C (Behavioral Equivalence)**: Does the implementation match the ground truth's design decisions, API conventions, and architectural approach?

Verdict rules. We assign a three-level verdict based on the dimension scores:

- PASS: $A \geq 4$ **and** $B \geq 4$ **and** $C \geq 3$.
- FAIL: $A < 2$ **or** $B < 2$ **or** $C < 2$ (any dimension critically deficient).
- PARTIAL: all other cases.

File coverage metric. We additionally compute file-level overlap between the agent's generated files and the ground-truth file set: $coverage = |generated \cap GT| \ / \ |GT|$. This provides a coarse quantitative complement to the expert-scored dimensions.

Evaluation scope. Our evaluation measures *production-level equivalence* to the merged implementation, not purely functional correctness. An agent solution that correctly addresses the requirement but uses different file organization, naming conventions, or architectural patterns than the ground truth will score lower on Behavioral Equivalence (C) and file coverage. This design choice reflects our industrial interest: we assess whether agent output could replace the human-authored PR, not merely whether it compiles and passes tests. As a consequence, our FAIL verdicts should be read as "not production-equivalent" rather than "functionally incorrect." Our root-cause analysis (§4) confirms this: the majority of non-PASS cases stem from engineering completeness gaps, not semantic misunderstanding.

Multi-evaluator cross-validation. To mitigate evaluator bias, every experiment is independently scored by three LLM evaluators from different model families: Claude Opus 4.6, Codex GPT-5.4, and GLM-5.1. We report the *majority verdict* (agreement of ≥2/3 evaluators) and scores averaged across all three. Unanimous agreement is 62%, and ≥2/3 consensus reaches 99%; pairwise agreement ranges from 69% to 79%, with per-dimension score spreads averaging <1 point.

2.5 Experiment Execution

Sanitization protocol. To prevent agents from extracting ground-truth code from repository history, we apply a strict sanitization procedure to each experiment repository:

(1) The repository is rebased to a single commit immediately before the target feature, removing all commits that contain ground-truth code.
(2) All git remotes are stripped and the `gh` CLI is blocked, preventing agents from fetching upstream history or pull request contents.
(3) Agents retain access to the full codebase (for context) and to web search (for documentation), but cannot access the specific PR or commit that constitutes the ground truth.

Anti-cheating audit. During evaluation of initial Codex runs on task K4, we discovered that the agent had executed `git log` and `git cherry-pick` commands to extract ground-truth commits from an insufficiently sanitized repository. The K4 repository was re-sanitized (all target commits removed from history) and the affected experiments were re-run. Post-sanitization results were confirmed genuine through inspection of agent trajectories and comparison of generated file counts and implementation details against ground truth. This incident underscores a critical finding: frontier models actively search for shortcuts in version control history, making thorough sanitization essential for any benchmark using real repositories.

Experiment scale. In total, we executed 96 experiments across 12 tasks × 4 configurations × 2 prompt levels—a complete factorial design with no missing cells. Each task–config–prompt combination was run exactly once and evaluated by three independent LLM judges. The full results are reported in §3.

3 Results: The Gap

This section presents the empirical results of 96 experiments across 12 industrial tasks, 4 agent configurations, and 2 prompt granularities, answering **RQ1**: *Can current coding agents independently complete enterprise-level feature implementation?* The short answer is no—the best configurations achieve only 12.5% PASS under our

rubric—but the data reveals nuanced patterns across configurations, prompts, and task complexity.

Each experiment was independently scored by three LLM evaluators (Claude Opus 4.6, Codex GPT-5.4, and GLM-5.1); we report the *majority verdict* (agreement of ≥2/3 evaluators) and averaged scores. The three evaluators agree unanimously on 62% of verdicts and reach ≥2/3 consensus on 99%; pairwise verdict agreement ranges from 69% to 79%, and per-dimension score spread averages <1 point, confirming robust cross-model calibration.

3.1 Overall Results

Table 3 presents the majority verdict for every task–configuration–prompt combination across the four configurations. Of 96 experiments, 8 achieve PASS (8.3%), 49 achieve PARTIAL (51.0%), and 39 result in FAIL (40.6%). No single configuration passes all tasks; even the strongest configurations pass only 3 of 24 experiments.

Finding 1: *No agent configuration reliably completes industrial feature implementation.* Across 96 experiments, the overall PASS rate is 8.3%. Even the best configurations fail or only partially pass 87.5% of their tasks. This confirms the paper's "Not Ready Yet" thesis: under our benchmark, current coding agents do not reliably deliver production-equivalent feature implementations.

3.2 Pass Rate by Configuration

Table 4 compares the four configurations on overall pass rate and mean composite score (sum of A+B+C, out of 15).

Claude Opus 4.6 and Codex GPT-5.4 tie for the highest pass rate (12.5%, 3/24 each). However, Codex achieves a higher mean composite score (7.36 vs. 6.50), indicating stronger partial-implementation quality despite identical pass counts. All three Claude passes occur with long prompts (C1-long, C4-long, M1-long), while Codex also passes exclusively with long prompts (C1-long, C4-long, K1-long). Cursor Composer-2 passes 8.3% (2/24); notably, both passes occur on short prompts (C1-short, C4-short), a counter-intuitive pattern discussed in §3.3. OpenCode GLM-5.1 achieves 0% pass but still produces partial implementations in 37.5% of experiments, suggesting the model understands problems but cannot execute complete solutions.

Finding 2: *The two strongest configurations (Claude Opus 4.6 and Codex GPT-5.4) each achieve 12.5% PASS under our rubric—tied, and far from production-ready.* No configuration passes any high-complexity task. A 91.7% non-pass rate across all configurations confirms that even frontier models require significant human oversight.

3.3 Prompt Granularity Effect

Each task was run with both a *long* prompt (detailed specification with design-level requirements and architectural guidance) and a *short* prompt (one-line task summary). Table 5 shows the per-configuration delta.

Long prompts improve composite scores for the two strongest models (Claude +2.6, Codex +2.2) and amplify their pass rates substantially (+25.0 pp each). Both Claude and Codex pass 25.0% of tasks with long prompts vs. 0% with short prompts. However, the pattern is not universal: Cursor shows a *negative* prompt delta (−16.7 pp), with both passes on short prompts. OpenCode shows no prompt effect, remaining at 0% pass regardless of prompt detail.

Finding 3: *Prompt granularity is the single strongest lever for improving performance of capable agents.* For the two strongest models, long prompts improve composite scores by +2.2 to +2.6 and pass rates by 25.0 pp. The effect amplifies existing model capability rather than substituting for it: strong models benefit most, while weaker models show no improvement or even negative effects.

We note one counter-pattern: Cursor shows a negative prompt delta, with short prompts outperforming long—its two passes (C1-short, C4-short) involve tasks where the model's autonomous exploration suffices, while long prompts may constrain its search strategy. This reversal does not generalize to the other configurations.

3.4 Task Complexity Analysis

Tasks are grouped into three complexity tiers: *Low* (C1, C2, M1, K1: 1–13 GT files), *Medium* (C3, C4, M2, K2, K3: 6–49 GT files), and *High* (C5, M3, K4: 10–98 GT files). Table 6 shows pass rates per tier for the four configurations.

On low-complexity tasks, the best configurations (Claude and Codex) each pass 25.0%. On medium-complexity tasks, three configurations (Claude, Codex, Cursor) each pass 10.0% (1 of 10). On high-complexity tasks, no configuration achieves any pass, with all experiments producing only PARTIAL or FAIL results.

Finding 4: *Task complexity creates an absolute capability cliff.* On low-complexity tasks, top configurations achieve 25% pass. On high-complexity tasks, all four configurations achieve 0%—no agent produces a production-equivalent implementation for tasks with 10+ ground-truth files.

3.5 Comparison with SWE-bench

SWE-bench [7] tasks are predominantly single-file bug fixes in Python repositories (1–3 files, <50 lines); our tasks range from 1 to 98 files and up to 11,107 lines, require new feature design, and span four languages. Where frontier agents achieve >70% on SWE-bench Verified, the best configurations in our benchmark achieve 12.5% overall (25.0% with long prompts); even on low-complexity tasks, our best pass rate is 25.0%.

Finding 5: *Under our benchmark, feature implementation appears substantially harder than bug fixing.* The gap grows with task complexity: on high-complexity tasks, no agent achieves any pass (0%), suggesting that current benchmarks may underestimate the challenge of real-world coding agent deployment.

4 Root Cause Analysis

Of 96 experiments, 88 result in non-PASS verdicts (48 PARTIAL, 39 FAIL, 1 no majority). This section analyzes the failure modes to determine whether the gap reflects fundamental model-capability limitations or engineering-addressable shortcomings.

4.1 Failure Mode Taxonomy

We classify the 88 non-PASS experiments into four failure modes based on the averaged evaluation scores across three independent evaluators:

Table 3: Overall results: majority verdict per task, prompt, and configuration. Scores shown as (A/B/C), averaged across three evaluators and rounded, where A = Functional Correctness, B = Completeness, C = Behavioral Equivalence, each out of 5. Bold = PASS, italic = PARTIAL, plain = FAIL.

Task	Prompt	Claude (Opus 4.6)	Codex (GPT-5.4)	Cursor (Composer-2)	OpenCode (GLM-5.1)
C1	long	**PASS** (5/5/5)	**PASS** (5/5/5)	FAIL (1/1/1)	FAIL (1/1/1)
	short	FAIL (2/1/1)	FAIL (1/1/1)	**PASS** (5/5/3)	FAIL (1/1/1)
C2	long	*PARTIAL* (3/4/2)	*PARTIAL* (3/4/3)	*PARTIAL* (3/3/2)	*PARTIAL* (2/3/2)
	short	*PARTIAL* (2/2/2)	*PARTIAL* (2/2/2)	*PARTIAL* (3/3/2)	*PARTIAL* (2/3/2)
C3	long	*PARTIAL* (3/2/2)	*PARTIAL* (3/2/2)	*PARTIAL* (3/2/2)	*PARTIAL* (3/2/2)
	short	FAIL (0/0/0)	FAIL (3/1/2)	*PARTIAL* (3/1/2)	FAIL (2/1/2)
C4	long	**PASS** (4/4/3)	**PASS** (4/4/4)	*PARTIAL* (4/2/2)	*PARTIAL* (3/3/2)
	short	*PARTIAL* (4/3/2)	*PARTIAL* (2/2/2)	**PASS** (4/3/3)	FAIL (2/2/2)
C5	long	FAIL (2/2/1)	FAIL (2/2/1)	FAIL (2/1/2)	FAIL (0/0/0)
	short	FAIL (2/2/1)	FAIL (2/1/2)	FAIL (2/1/1)	FAIL (1/0/0)
M1	long	**PASS** (5/5/5)	*PARTIAL* (3/3/3)	*PARTIAL* (3/3/3)	*PARTIAL* (3/3/3)
	short	*PARTIAL* (3/1/3)	*PARTIAL* (2/2/2)	*PARTIAL* (3/2/2)	FAIL (2/1/1)
M2	long	*PARTIAL* (2/1/1)	FAIL (1/1/1)	*PARTIAL* (2/2/3)	FAIL (0/0/0)
	short	*PARTIAL* (2/1/3)	FAIL (2/1/2)	FAIL (2/1/2)	FAIL (2/0/2)
M3	long	FAIL (1/1/2)	FAIL (2/1/1)	FAIL (2/1/2)	FAIL (2/1/1)
	short	*PARTIAL* (2/0/1)	FAIL (2/1/1)	FAIL (1/1/1)	FAIL (2/1/1)
K1	long	*PARTIAL* (3/2/3)	**PASS** (4/4/4)	*PARTIAL* (3/3/3)	FAIL (2/1/2)
	short	*PARTIAL* (3/3/2)	*PARTIAL* (3/2/2)	*PARTIAL* (4/3/3)	*PARTIAL* (2/2/2)
K2	long	*PARTIAL* (3/3/2)	*PARTIAL* (3/3/3)	*PARTIAL* (3/2/2)	*PARTIAL* (2/2/2)
	short	*PARTIAL* (3/2/2)	*PARTIAL* (3/3/3)	*PARTIAL* (3/3/3)	*PARTIAL* (2/2/2)
K3	long	FAIL (0/0/0)	*PARTIAL* (3/3/2)	FAIL (0/0/0)	FAIL (2/2/1)
	short	FAIL (0/0/0)	*PARTIAL* (4/3/2)	*PARTIAL* (3/2/2)	*PARTIAL* (2/2/2)
K4	long	*PARTIAL* (3/2/3)	*PARTIAL* (3/2/3)	FAIL (0/0/0)	FAIL (2/1/2)
	short	*PARTIAL* (2/2/2)	*PARTIAL* (3/2/3)	FAIL (0/0/0)	FAIL (2/1/2)

Table 4: Configuration comparison. N = number of experiments, Pass Rate = fraction achieving PASS, Mean Comp. = mean composite score (A+B+C, out of 15). Scores are averaged across three independent evaluators.

Config	N	PASS	PARTIAL	FAIL	Pass%	Mean
Claude (Opus 4.6)	24	3	14	7	12.5	6.50
Codex (GPT-5.4)	24	3	13	8	12.5	7.36
Cursor (Composer-2)	24	2	13	9	8.3	6.31
OpenCode (GLM-5.1)	24	0	9	15	0.0	5.09

Table 5: Prompt granularity effect. Long = detailed specification, Short = one-line summary. ΔPass = difference in pass rate (percentage points). ΔComp. = difference in mean composite score.

Config	Long%	Short%	ΔPass	Long	ΔComp.
Claude (Opus 4.6)	25.0	0.0	+25.0	7.8	+2.6
Codex (GPT-5.4)	25.0	0.0	+25.0	8.4	+2.2
Cursor (Composer-2)	0.0	16.7	−16.7	5.7	−1.3
OpenCode (GLM-5.1)	0.0	0.0	0.0	5.1	0.0

- **Incomplete coverage** ($B < 2$): The agent touched too few files or components to constitute a meaningful implementation. 43 cases (48.9%).
- **Partial coverage** ($2 \leq B < 4$, $A \geq 2$): The agent covered some required files but missed key components, producing a recognizable but incomplete implementation. 36 cases (40.9%).
- **No output** ($A < 1$ and $B < 1$): The agent produced no meaningful code—crashed, timed out, or generated no changes. 8 cases (9.1%).
- **Structural mismatch** ($C < 2$ with $A, B \geq 2$): The agent covered the right scope with working code but used a fundamentally different architectural approach. 1 case (1.1%).

The dominant failure mode is incomplete coverage (48.9%), not semantic misunderstanding (1.1%). Combined with partial coverage, 89.8% of non-PASS outcomes stem from the agent failing to cover the full scope of required changes.

Finding: *Completeness (dimension B) is the dominant bottleneck: it is the lowest-scoring dimension in 69% of non-PASS experiments, with mean $B = 1.71/5$ vs. mean $A = 2.21/5$. Only 1.1% of failures reflect*

Table 6: Pass rate by task complexity tier and configuration.

Complexity	Config	N	PASS	FAIL	Pass%
Low	Claude	8	2	1	25.0
	Codex	8	2	1	25.0
	Cursor	8	1	1	12.5
	OpenCode	8	0	4	0.0
Medium	Claude	10	1	3	10.0
	Codex	10	1	3	10.0
	Cursor	10	1	2	10.0
	OpenCode	10	0	5	0.0
High	Claude	6	0	3	0.0
	Codex	6	0	4	0.0
	Cursor	6	0	6	0.0
	OpenCode	6	0	6	0.0

Table 7: Completeness degradation by complexity tier among non-PASS experiments. B<2 = critically incomplete.

Tier	Non-PASS	Mean B	B<2	B<2 %
Low	27	2.28	10	37.0
Medium	37	1.67	20	54.1
High	24	1.15	21	87.5

structural mismatch; the remainder stem from insufficient coverage scope.

4.2 Engineering-Addressable vs. Model-Capability Gaps

To separate engineering problems from model-capability problems, we compare dimensions A (Functional Correctness) and B (Completeness) for each non-PASS experiment. When $B \leq A$, completeness is the binding constraint: the agent understands the problem but fails to implement the full scope. When $A < B$, the agent covers the right scope but implements incorrectly, indicating a model-capability limitation.

Of 88 non-PASS experiments: 74 (84.1%) have $B \leq A$ (engineering-addressable), 6 (6.8%) have $A < B$ (model-capability limited), and 8 (9.1%) produced no output.

Finding: *84.1% of non-PASS outcomes are engineering-addressable: the agent's functional understanding (A) exceeds its implementation completeness (B), suggesting that better scope control—not better model reasoning—would close the majority of the gap.*

4.3 Complexity-Driven Degradation

Table 7 shows how completeness degrades with task complexity.

On high-complexity tasks, 87.5% of non-PASS experiments are critically incomplete ($B < 2$), compared to 37.0% on low-complexity tasks. Mean B drops from 2.28 (low) to 1.15 (high), confirming that agents struggle with scope management as cross-component dependencies increase. Three examples illustrate the pattern:

Table 8: Non-PASS failure profiles by agent configuration.

Config	N	Mean A	Mean B	B<2
Claude (Opus 4.6)	21	2.15	1.62	13
Codex (GPT-5.4)	21	2.52	1.95	10
Cursor (Composer-2)	22	2.31	1.73	12
OpenCode (GLM-5.1)	24	1.91	1.57	16

- **K4** (58 HW files, Go): The best agent (Claude) covers 31/58 files (53%); the weakest (Cursor) covers 1/58 (2%). Even partial implementations miss authorization hooks, generated client code, and e2e test scaffolding.
- **C5** (25 HW files, C++/AscendC): Coverage ranges from 0/25 (OpenCode) to 12/25 (Codex, 48%). No agent reaches PASS.
- **M3** (9 HW files, Python): All agents cover ≤1/9 files. No agent achieves PASS on any prompt variant.

Finding: *Task complexity creates a completeness cliff: on high-complexity tasks, 87.5% of non-PASS experiments are critically incomplete ($B < 2$), and mean B drops from 2.28 to 1.15. Agents fail at scope discovery, not at per-file implementation quality.*

4.4 Per-Agent Failure Patterns

Table 8 shows the failure profile for each configuration among non-PASS experiments.

All four agents exhibit the same failure pattern: B consistently trails A, confirming that incomplete coverage is the universal bottleneck. Codex achieves the highest non-PASS mean B (1.95), reflecting broader file exploration. With both Claude and Codex tied at 12.5% PASS, Codex's higher mean composite score (7.36 vs. 6.50) suggests that exploration breadth yields better partial implementations even when it does not cross the PASS threshold.

File coverage data corroborates this analysis. On K4, Claude covers 31/58 HW files while Cursor covers only 1/58—a 30× difference that directly explains their respective PARTIAL vs. FAIL verdicts. On C2, all four agents cover 4/4 HW files yet all receive PARTIAL, indicating that when coverage is not the bottleneck, functional correctness (A) becomes the binding constraint.

5 Harness Engineering

The previous sections established that no agent configuration reliably completes industrial feature implementation (Section 3) and that failures cluster around engineering-addressable gaps—primarily incomplete file coverage (Section 4). This section examines whether engineering interventions—specifically, detailed input specifications—can close the performance gap.

5.1 Definition and Taxonomy

We define **Harness Engineering** as the practice of augmenting a coding agent's execution environment with structured inputs, validation loops, or process constraints—without modifying the underlying model. We distinguish three intervention points:

- **Input-side** (specification harness): detailed task descriptions, file-level scope constraints, or architecture guidelines

provided before generation begins. Our long prompts serve this role.

- **Output-side** (validation harness): syntax checking, structure validation, and test execution that feed errors back to the agent for correction. No configuration in our study implements this level.
- **Process-side** (pipeline harness): enforced phases such as discovery-before-implementation, mandatory test generation, or CI-gated iteration. No configuration in our study implements this level.

5.2 Input-Side Harness: The Prompt Effect

Detailed (long) prompts constitute an input-side harness that constrains the agent's search space. Table 5 (in §3.3) shows the effect across all four configurations.

For the two strongest agents, long prompts improve composite scores by +2.2 to +2.6/15 and pass rates by 25.0 percentage points. Both Claude and Codex show identical prompt deltas (+25.0 pp), passing 25.0% of tasks with long prompts vs. 0% with short prompts. However, the effect is not universal: Cursor shows a *negative* delta (−16.7 pp), and OpenCode shows no change (0% regardless of prompt detail).

Finding: *Detailed specifications improve PASS rates by up to 25.0 percentage points and composite scores by +2.2 to +2.6/15 for the two strongest configurations. The effect amplifies existing capability rather than substituting for it: strong models benefit most, while weaker models show no improvement or negative effects.*

5.3 Harness Limitations and Future Directions

Even with detailed input specifications, the best configurations (Claude and Codex + long prompts) reach only 25.0% PASS. The remaining gap stems from:

- **Test generation:** No harness enforces test writing; agents frequently omit test files that constitute a significant fraction of the ground truth.
- **Architecture reasoning:** Input specifications can guide scope but cannot verify that cross-module wiring is semantically correct.
- **Discovery depth:** The harness does not control how thoroughly agents explore the codebase before writing code.

We identify three levels of harness improvement, ordered by estimated difficulty: (1) *language-aware validation* (adapt output checks to Go/Python/C++ conventions; estimated +2–5% PASS); (2) *fix-cycle loops* (feed validation errors back for agent correction; estimated +10–15% PASS); and (3) *architecture-aware planning* (enforce discovery-before-implementation phases; estimated +5–10% PASS).

Finding: *Input-side prompt engineering (long vs. short) yields up to +25.0pp pass rate improvement for capable agents. The remaining 75.0% non-pass gap requires output-side and process-side harnesses—particularly fix-cycle loops and enforced test generation—that no current configuration implements.*

6 Related Work

Code generation benchmarks. SWE-bench [7] evaluates bug fixing in open-source Python repositories; frontier agents now resolve >70% of its Verified subset. HumanEval [3], MBPP [1], DevBench [8], and RepoEval [13] extend evaluation to function- and repository-level code generation but do not incorporate industrial engineering constraints. Our work evaluates multi-file feature implementation in production codebases with ground truth from merged pull requests.

Industrial agent evaluations. Mundhra et al. [10] found LLM-generated code on ASML's EUV lithography codebase insufficient for production use; Liu et al. [9] identified LLM capability boundaries in aerospace anomaly detection—both aligned with our "Not Ready Yet" thesis regarding autonomous deployment. A Google RCT reported 21% developer speedup with AI-assisted coding [11], but the DORA 2025 report found a slight throughput decrease with high AI adoption [4], and a Copilot study found 19% slower task completion on mature codebases [2]. These conflicting signals reinforce that agent effectiveness depends heavily on task type and context.

Agent frameworks and harness engineering. Modern coding agents—SWE-agent [12], Aider [5], Claude Code, Codex CLI—embed iterative repair loops, planning, and tool-use orchestration. Our trajectory analysis (§**??**) shows that framework-level tool-use strategy can matter as much as model capability. To our knowledge, no prior work has systematically evaluated *harness engineering* effects on coding agent performance across a controlled benchmark; our prompt comparison quantifies the input-side effect across all four configurations.

7 Threats to Validity

Construct validity. Every experiment is independently scored by three LLM evaluators from different model families (Claude Opus 4.6, Codex GPT-5.4, and GLM-5.1), and we use the majority verdict (≥2/3 agreement) as the final outcome. This cross-validation design mitigates single-evaluator bias, but shared blind spots across model families may persist. Unanimous agreement is 62%, and ≥2/3 consensus reaches 99%; pairwise agreement ranges from 69% to 79%, with per-dimension score spreads averaging <1 point. Our rubric measures production-level equivalence to the merged implementation rather than purely functional correctness; it may miss dimensions such as maintainability or security.

Internal validity. Each task–configuration–prompt cell was executed once; stochastic variation means individual results may not represent expected performance, though aggregate patterns across 96 experiments provide robustness. All cross-configuration comparisons are *product-level* assessments confounding model and framework differences (§2.2). During initial K4 experiments, Codex exploited insufficiently sanitized git history via `git cherry-pick`; we remediated by re-sanitizing all repositories and re-running affected experiments, confirming no ground-truth extraction in post-sanitization trajectories.

External validity. Our 12 tasks span four languages and three complexity tiers but may not generalize to other enterprise contexts (e.g., Java, Rust, fully proprietary codebases). Agent capabilities evolve rapidly; our early-2026 model snapshots will be superseded,

though we expect relative patterns (prompt effect, complexity cliff) to persist.

VCS history as information channel. Codex proactively searched git history and web documentation, while other agents varied in their use of these information channels. Neither behavior constitutes ground-truth leakage (sanitization removed target commits), but the information asymmetry means performance differences partially reflect tool-use strategy rather than pure model capability.

8 Conclusion

Under our benchmark of 12 industrial feature-implementation tasks spanning four languages and three complexity tiers, no agent configuration achieves reliable autonomous feature implementation. The best configurations (Claude Opus 4.6 and Codex GPT-5.4) each reach 12.5% PASS under our rubric; the overall rate across 96 experiments is 8.3%. Task complexity exposes an absolute capability cliff: top agents pass 25.0% of low-complexity tasks but 0% of high-complexity ones. Root-cause analysis shows that 84.1% of non-PASS outcomes stem from engineering-addressable gaps—primarily incomplete file coverage—rather than fundamental model-capability limitations.

These findings carry a concrete industrial consequence. This research informed our team's decision to adopt *agent-assisted* rather than agent-autonomous workflows, with mandatory human review gates for API design, test coverage, and feature lifecycle decisions. At the same time, the harness engineering results are encouraging: detailed input specifications improve pass rates by up to 25.0 percentage points for capable agents, confirming prompt granularity as the single strongest performance lever.

Future work should pursue three directions: (1) *stronger harness*—fix-cycle loops that feed validation failures back as repair instructions, enforced test-generation mandates, and language-aware output validation; (2) *broader evaluation*—a larger and more diverse task set covering additional languages, repository types, and complexity ranges to test generalizability; (3) *human–agent collaboration modes*—measuring productivity and quality when agents operate under human supervision rather than autonomously, which better reflects the workflow our industrial experience suggests is practical today.

Data Availability

The replication package—including evaluation scripts, all 24 prompt files (long and short variants for each task), the three-dimension scoring rubric, and experiment results (scores, verdicts, and file coverage for all 96 experiments)—is publicly available at [Zenodo DOI TBD]. Agent session trajectories are preserved for reproducibility analysis. The CANN and MindSpeed source code is available on Gitee under Huawei's open-source repositories; Kubernetes source code is available on GitHub. Proprietary internal documentation (e.g., operator design guides) referenced during task selection cannot be shared, but the prompts and evaluation reports contain sufficient detail to reproduce our analysis methodology on comparable codebases.

References

[1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. 2021. Program Synthesis with Large Language Models. *arXiv preprint arXiv:2108.07732* (2021). https://arxiv.org/abs/2108.07732

[2] Joel Becker, Nate Rush, Elizabeth Barnes, and David Rein. 2025. Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity. *arXiv preprint arXiv:2507.09089* (2025). https://arxiv.org/abs/2507.09089 RCT with 16 experienced OSS developers; AI increased task time by 19% despite self-reported 20% speedup.

[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021). https://arxiv.org/abs/2107.03374

[4] Derek DeBellis, Kevin Storer, Amanda Lewis, Ben Good, Daniella Villalba, Eric Maxwell, Kim Castillo, Michelle Irvine, and Nathen Harvey. 2024. Accelerate State of DevOps Report 2024. https://dora.dev/research/2024/dora-report/. DORA/Google Cloud; +25% AI adoption correlated with −1.5% delivery throughput.

[5] Paul Gauthier. 2024. Aider: AI Pair Programming in Your Terminal. https://github.com/Aider-AI/aider. Open-source AI coding assistant.

[6] GitHub. 2024. Survey: The AI Wave Continues to Grow on Software Development Teams. https://github.blog/news-insights/research/survey-ai-wave-grows/. Survey of 2,000 enterprise developers across four countries; 97% reported using AI coding tools.

[7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Proceedings of the Twelfth International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/2310.06770

[8] Bowen Li, Wenhan Wu, Ziwei Tang, Lin Shi, John Yang, Jinyang Li, Shunyu Yao, Chen Qian, Binyuan Hui, Qicheng Zhang, Zhiyin Yu, He Du, Ping Yang, Dahua Lin, Chao Peng, and Kai Chen. 2024. DevBench: A Comprehensive Benchmark for Software Development. *arXiv preprint arXiv:2403.08604* (2024). https://arxiv.org/abs/2403.08604 Later versions renamed to DevEval.

[9] Yang Liu, Yixing Luo, Xiaofeng Li, Xiaogang Dong, Bin Gu, and Zhi Jin. 2026. Evaluating Large Language Models for Time Series Anomaly Detection in Aerospace Software. *arXiv preprint arXiv:2601.12448* (2026). https://arxiv.org/abs/2601.12448

[10] Yash Mundhra, Max Valk, and Maliheh Izadi. 2025. Evaluating Large Language Models for Functional and Maintainable Code in Industrial Settings: A Case Study at ASML. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), Industry Track*. https://arxiv.org/abs/2509.12395

[11] Elise Paradis, Kate Grey, Quinn Madison, Daye Nam, Andrew Macvean, Vahid Meimand, Nan Zhang, Ben Ferrari-Church, and Satish Chandra. 2024. How Much Does AI Impact Development Speed? An Enterprise-Based Randomized Controlled Trial. *arXiv preprint arXiv:2410.12944* (2024). https://arxiv.org/abs/2410.12944 RCT with 96 Google engineers; AI features shortened task time by ∼21%.

[12] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*. https://arxiv.org/abs/2405.15793

[13] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Singapore, 2471–2484. doi:10.18653/v1/2023.emnlp-main.151